

BFS

$O(n + m)$

```
for i = 1, n  
    viz[i] = 0  
    dist[i] = ∞  
    tata[i] = 0  
    }
```

$O(n)$

BFS(s)

wada $\ell = \emptyset$

adauga(s, ℓ)

viz[s] = 1

tata[s] = 0; dist[s] = 0;

cat timp $\ell \neq \emptyset$ execută

nod = extrage(ℓ) $\rightarrow O(1)$

afiseaza(nod)

for $vein \in L(nod)$

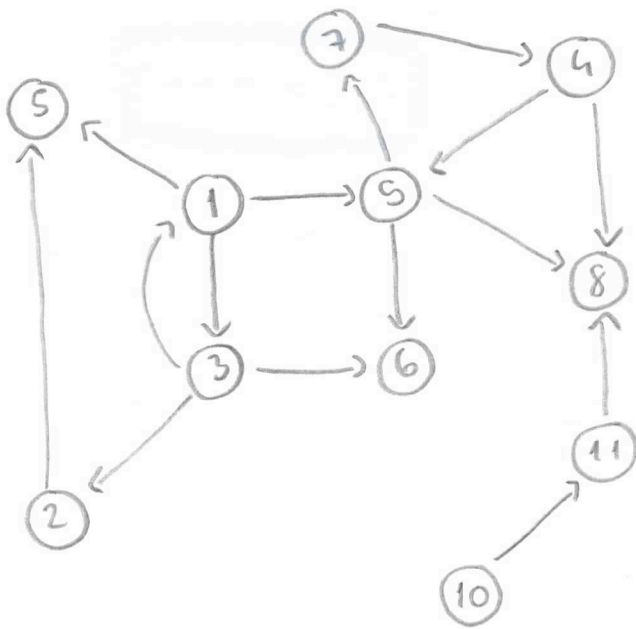
if (viz[vein] == 0)

adauga(vein, ℓ)

viz[vein] = 1;

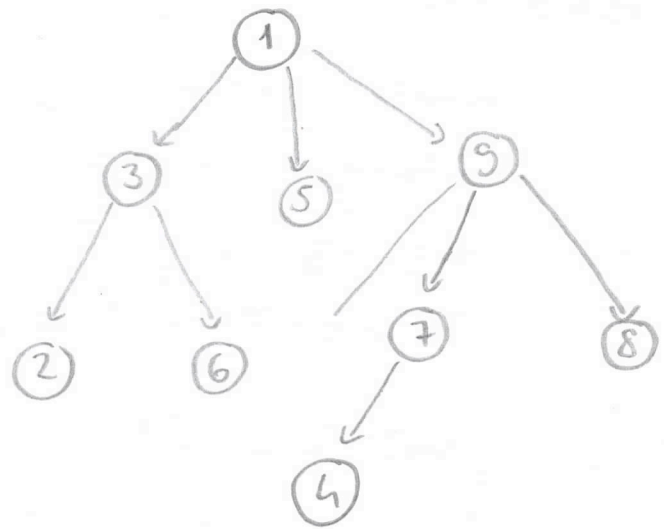
tata[vein] = nod

dist[vein] = dist[nod] + 1



BFS (1)

C: ~~1~~ ~~3~~ ~~5~~ ~~7~~ 2 6
~~4~~ ~~8~~ 11



1	2	3	4	5	6	7	8	9	10	11
0	3	1	7	1	3	9	9	1	∞	∞

lala

1	2	3	4	5	6	7	8	9	10	11
0	2	4	3	1	2	2	2	4	0	0

dist

DFS

DFS(x)

unloare[x] = gr

time = time + 1

desc[x] = time

for vein ∈ L[x]

if (unloare[vein] == alb)

tata[vein] = x;

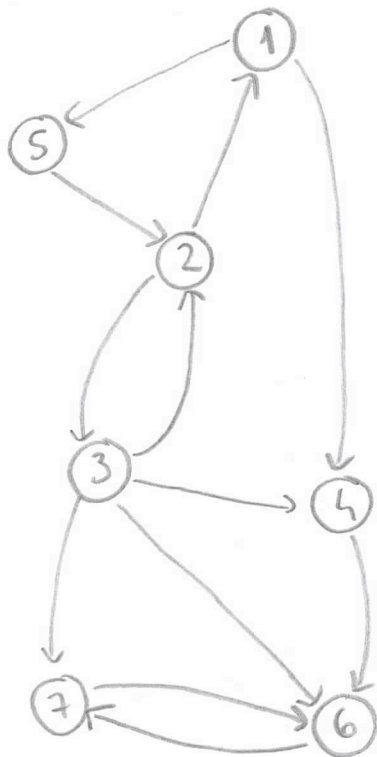
dist[vein] = dist[x] + 1

DFS(vein)

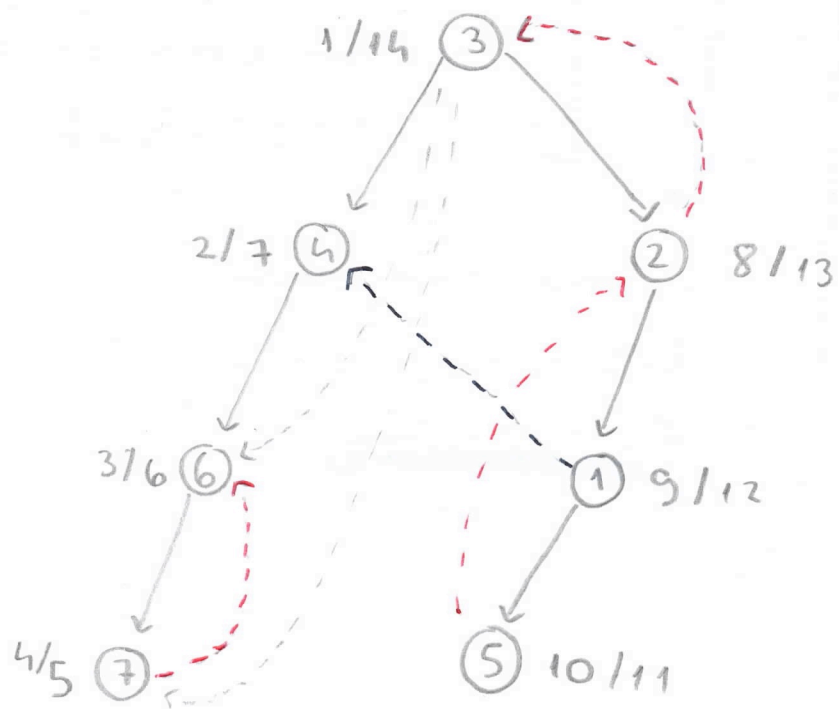
unloare[x] = negru

time = time + 1

fin[x] = time



DFS (3)



mediu (7,6) este de intrare și este dex.
 pentru ca nodul 6 este iniț. și

$$\text{dex}[6] < \text{dex}[7] < \text{fin}[7] < \text{fin}[6]$$

mediu

SORTARE TOPOLOGICA

① wada $l = \emptyset$

adaugam toate $v \in V$ in $d^-[v] = 0$

cat timp $l \neq \emptyset$

$nod = \text{extrage}(l)$

 adauga nod la sortare

 for $vein \in L(nod)$

$d^-[vein] = d^-[vein] - 1$

 if ($d^-[vein] == 0$)

 adauga($vein, l$)

②

wada $l = \emptyset$

DFS(x)

$viz[x] = 1$

 for $vein \in L(x)$

 DFS($vein$)

 // x este finalizat (negru)

 adauga($l, vein$)

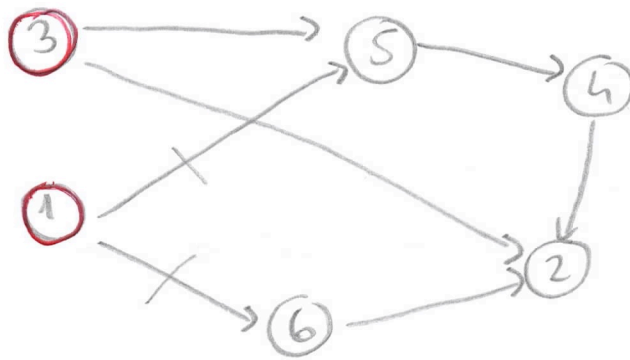
for $v \in V$

 DFS(v)

while $l \neq \emptyset$

$nod = \text{extrage}(l)$

 adauga nod la sortare

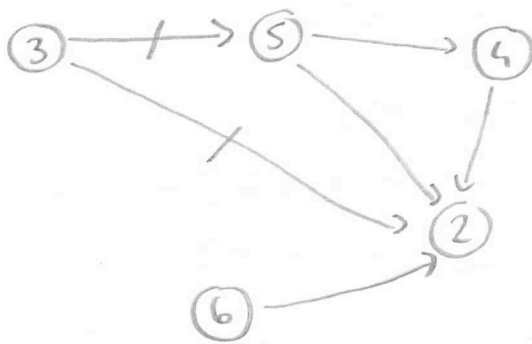


$C: 1 \ 3 \ 6 \leq 4$

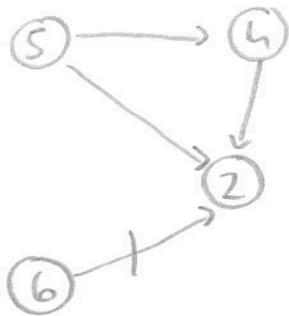


$C: \underline{1} \ 3 \ 6$

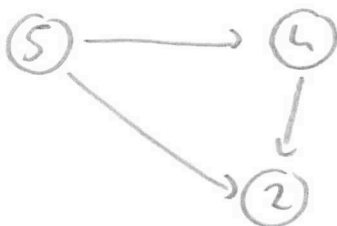
$C: 1 \ 3 \ 6 \ 5 \leq 4 \ 2$



$C: 1 \ \underline{3} \ 6 \ 5$



$C: 1 \ 3 \ \underline{6} \ 5$



KOSARAJU

DET. COMPONENTELOR TARE CONEXE

ada $\mathcal{C} = \emptyset$

DFS(G, x)

$viz[x] = 1$

for $vein \in L[x]$

if ($viz[vein] == 0$)

 DFS($G, vein$)

adauga($x, \mathcal{C}$)

for $nod \in V$

if ($viz[nod] == 0$)

 DFS(G, nod)

for $i = 1, n$

$viz[i] = 0$

while $\mathcal{C} \neq \emptyset$

$nod = extragere(\mathcal{C})$

if ($viz[nod] == 0$)

 DFS(G^T, nod)

 afisare comp. tare conexa

$G^T = G$ dar nuhile
nu e inversat

PRIM

DET. ARBORI PARTIALI DE COST MIN

pentru $u \in V$
 $d[u] = \infty$
 $tata[u] = 0$

Q se initializează cu V // $Q = V(G) - V(T)$
= vf. neselectate în APM

$d[s] = 0$

while $Q \neq \emptyset$

$u = \text{extrage un vf } \in Q \text{ cu } d \text{ minim}$

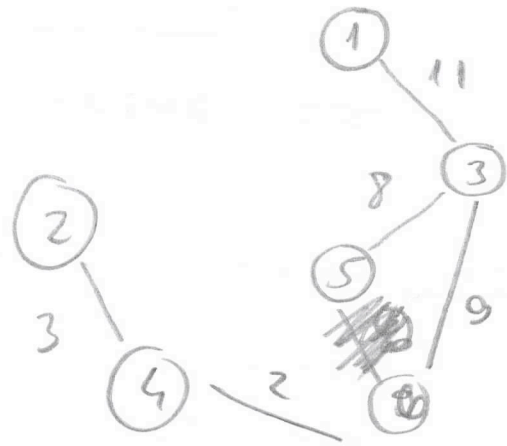
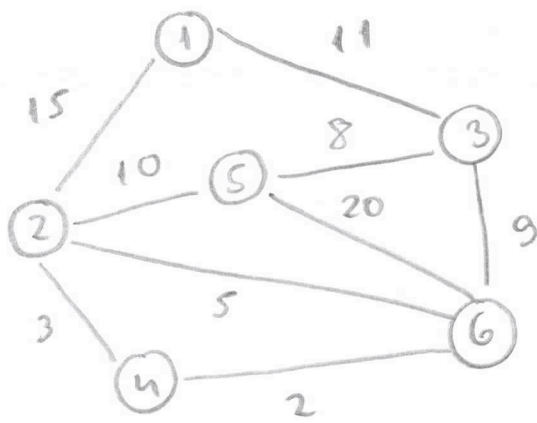
for $w \in E$ există

dacă $v \in Q$ și $w(u, v) < d[v]$

$d[v] = w(u, v)$

$tata[v] = u$

return $(u, tata[u])$ pentru $u \neq s$



d / lista

	1	2	3	4	5	6
	0/0	∞ /0	∞ /0	∞ /0	∞ /0	∞ /0

-	15/1	11/3	∞ /0	∞ /0	∞ /0
---	------	------	-------------	-------------	-------------

-	15/1	-	∞ /0	8/3	9/3
---	------	---	-------------	-----	-----

-	10/5	-	∞ /0	-	9/3
---	------	---	-------------	---	-----

-	5/6	-	2/6	-	-
---	-----	---	-----	---	---

-	3/4	-	-	-	-
---	-----	---	---	---	---

-	-	-	-	-	-
---	---	---	---	---	---

KRUSKAL

ARBORI PARTIALI DE COST MINIM

mergeala (E)

for $v \in V$
 initializeaza (v)

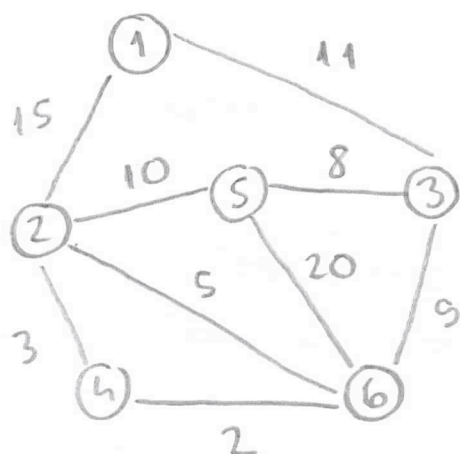
nr-nuchi-sel = 0

for $uv \in E$
 if (Repr(u) $\neq$ Repr(v))
 $E(T) = E(T) \cup \{uv\}$
 Reunește (u, v)
 nrmsel += 1
 if (nrmsel == n-1)
 STOP

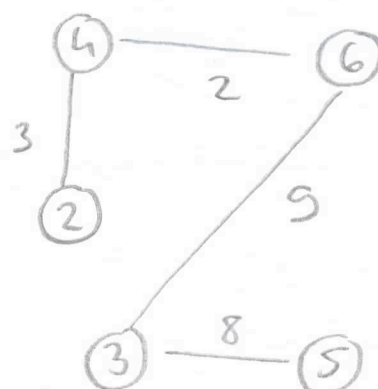
void Initialize (nod)
 tata[nod] = h[nod] = 0

int Reprezentant (nod)
 if (tata[nod] == 0)
 return nod comprisie
 tata[nod] = Repr (tata[nod])
 return tata[nod]

void Reunește (u, v)
 rv = Reprezentant (v)
 ru = Reprezentant (u);
 if (h[rv] > h[ru])
 tata[rv] = ru
 else
 tata[ru] = rv
 if (h[rv] == h[ru])
 h[rv] += 1



$[4, 6]$
 $[2, 4]$
 $[2, 6]$
 $[3, 5]$
 $[3, 6]$
 $[2, 5]$
 $[1, 3]$
 $[1, 2]$
 $[5, 6]$



$[4, 6]$

$n = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$

$n(4) \neq n(6) \Rightarrow \text{Remove}$

$[2, 4]$

$n = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 4 \\ 1 & 2 & 3 & 4 & 5 & 4 \end{bmatrix}$

$n(2) \neq n(4) \Rightarrow \text{Remove}$

$[2, 6]$

$n = \begin{bmatrix} 1 & 2 & 3 & 2 & 5 & 2 \\ 1 & 2 & 3 & 2 & 5 & 2 \end{bmatrix}$

$n(2) = n(6) \rightarrow \text{NO}$

$[3, 5]$

$n = \begin{bmatrix} 1 & 2 & 3 & 2 & 3 & 2 \\ 1 & 2 & 3 & 2 & 3 & 2 \end{bmatrix}$

$n(3) \neq n(5) \rightarrow \text{Remove}$

$[3, 6]$

$n = \begin{bmatrix} 1 & 2 & 3 & 2 & 3 & 2 \\ 1 & 2 & 3 & 2 & 3 & 2 \end{bmatrix}$

$n(3) \neq n(6) \rightarrow \text{Remove}$

$[2, 5]$

$n = \begin{bmatrix} 1 & 2 & 2 & 2 & 2 & 2 \\ 1 & 2 & 2 & 2 & 2 & 2 \end{bmatrix}$

$n(2) = n(5) \rightarrow \text{NO}$

$[1, 3]$

$n = \begin{bmatrix} 1 & 2 & 3 & 2 & 3 & 2 \\ 1 & 2 & 3 & 2 & 3 & 2 \end{bmatrix}$

$n(1) \neq n(3) \rightarrow \text{Remove}$

$n = \begin{bmatrix} 3 & 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 & 3 \end{bmatrix}$

CLUSTERING

GRUPAREA UNOR OBIECTE IN K CLASE

void Init()

for $i = 0, n-2$

for $j = i+1, n-1$ // construiim Gf complet

cost = Dist + Edit ($uov[i]$, $uov[j]$)

adauga muchia (i, j, cost) in E

for $i = 0, n-1$

h[i] = tata[i] = 0

int Find (u)

if $tata[u] == 0$

return u

tata[u] = ~~Find~~ Find (tata[u])

return tata[u]

void Union (u, v)

$ru = \text{Find}(u)$; $rv = \text{Find}(v)$;

if ($h[ru] > h[rv]$) $tata[rv] = ru$

else { $tata[ru] = rv$;

if ($h[rv] == h[ru]$) $h[rv]++$ }

return E

void Kruskal

$nr_clusters = n$

for $uv \in E$

if ($\text{Find}(u) \neq \text{Find}(v)$)

{ if ($nr_clusters > K$) { Union (u, v); $nr_clusters--$ }

; else { $grad_reparare = uv.cost$; BREAK; }


```
int dist-editare ( cuv1, cuv2, n, m)
```

```
{
```

```
    if m == 0
        return n
```

```
    elif n == 0
        return m
```

```
    if cuv1[n] == cuv2[m]
        dist-edit ( cuv1, cuv2, n-1, m-1)
```

```
    insereare = dist-edit ( cuv1, cuv2, n, m-1)
```

```
    delete = dist-edit ( cuv1, cuv2, n-1, m-1)
```

```
    replace = dist-edit ( cuv1, cuv2, n-1, m-1)
```

```
    return 1 + min ( insereare, delete, replace)
```

```
}
```

		n	e	s	t	a	n	t	a
	0	1	2	3	4	5	6	7	8
v	1	1	2	3	4	5	6	7	8
i	2	2	2	3	4	5	6	7	8
a	3	3	3	3	4	4	5	6	7
t	4	4	4	4	3	4	5	5	6
a	5	5	5	5	4	3	4	5	5

dist-edit = 5

DRUMURI MINIME DE SURSA UNICA ÎN DAG

$O(m+n)$

- graful nu conține cicluri
- arcele pot avea cost negativ

s - sf de start

// initializam distanțele

```
for  $v \in V$ 
   $dist[v] = \infty$ 
   $tata[v] = 0$ 
```

$O(n) (*) \rightarrow -\infty$

// det o sortare topologică

Sort Top = sortare-topologica(G) $\rightarrow O(n+m)$

pentru $u \in \text{Sort Top}$

pentru $uv \in E \rightarrow O(m)$

RELAXARE

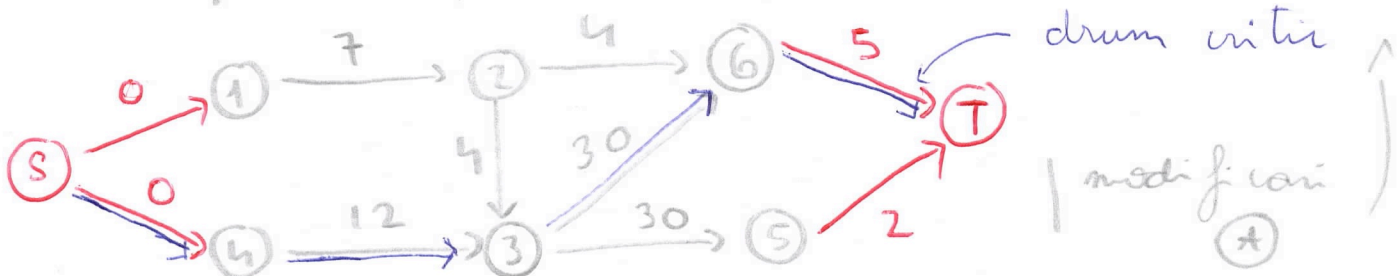
if $d[u] + w(u,v) < d[v]$ $(*)$
 $d[u] + w(u,v) > d[v]$

$d[v] = d[u] + w(u,v)$

$tata[v] = u$

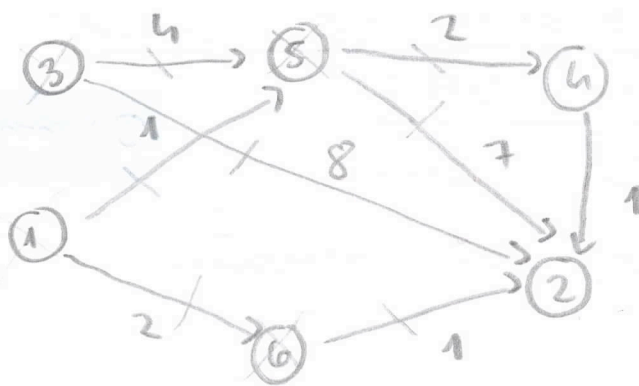
DRUMURI CRITICE

- un proiect cu n activități cu o durată de timp;
prechi (i,j) = proiectul i trebuie făcut înainte de cel j ; se pot
face în paralel timpul minim de realizare



lung min de realizare = costul maxim al unui drum de la S la T

drum critic = drum de cost maxim de la S la T



$n = 3$

Sortare Topologica : ~~1~~ ~~3~~ ~~6~~ ~~5~~ ~~4~~ ~~2~~

	1	2	3	4	5	6
d/tota	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$

$u = 1$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$
---------	------------	------------	------------	------------	------------	------------

$\infty + 4 < \infty$ NU
 $\infty + 2 < \infty$ NU

$u = 3$	$\infty/0$	8/3	0/0	$\infty/0$	4/3	$\infty/0$
---------	------------	-----	-----	------------	-----	------------

$0 + \infty < \infty$ DA
 $6 + 4 < \infty$ DA

$u = 6$	$\infty/0$	8/3	0/0	$\infty/0$	4/3	$\infty/0$
---------	------------	-----	-----	------------	-----	------------

$\infty + 1 < \infty$ NU

$u = 5$	$\infty/0$	8/3	0/0	6/5	4/3	$\infty/0$
---------	------------	-----	-----	-----	-----	------------

$4 + 2 < \infty$ DA
 $4 + 7 < 8$ NU

$u = 4$	$\infty/0$	7/4	0/0	6/5	4/3	$\infty/0$
---------	------------	-----	-----	-----	-----	------------

$6 + 1 < 8$ DA

$u = 2$	$\infty/0$	7/4	0/0	6/5	4/3	$\infty/0$
---------	------------	-----	-----	-----	-----	------------

DIJKSTRA

$O(m \log n)$

- arcele au costuri POZITIVE !
- poate avea cicluri

$\left[\begin{array}{l} \text{for } \text{nod} \in V \\ \text{dist}[\text{nod}] = \infty; \text{tata}[\text{nod}] = 0; \end{array} \right\} O(n)$

$d[s] = 0$

$PQ = V$ (MinHeap cu cost + nod)

while $PQ \neq \emptyset$

$u = \text{extract_min}(PQ) \rightarrow O(n \log n)$

for $uv \in E$

if $d[u] + w(u, v) < d[v]$

$d[v] = d[u] + w(u, v)$

repara $(PQ, v) \rightarrow O(m \log n)$

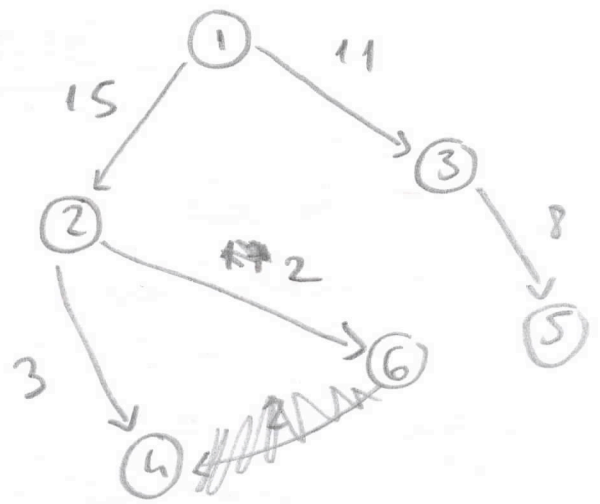
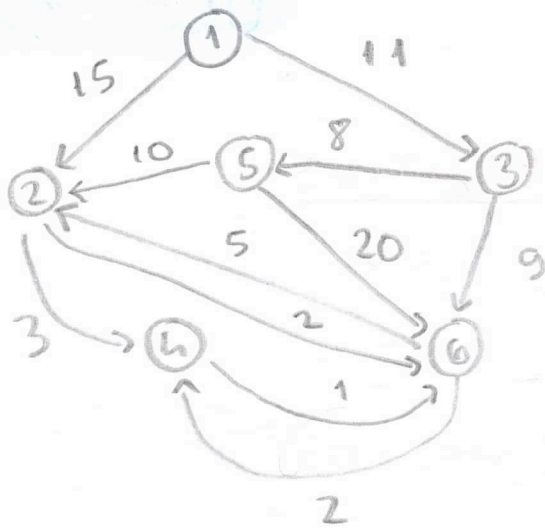
$tata[v] = u$

return $d, tata$

// are natura Greedy $\Rightarrow$ adangarea unei muchii la un drum existent nu poate duce la ~~reducere~~ creșterea sau se menține costul

// la fiecare pas alege nodul cu cea mai mică distanță estimată curentă, îl finalizează și NU îl mai întorcem $\Rightarrow$ un drum mai lung

poate fi rezultatul a interconectării tuturor nodurilor



d / total

	1	2	3	4	5	6
	0/0	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$

rel 1

	0/0	15/11	11/1	$\infty/0$	$\infty/0$	$\infty/0$
--	-----	-------	------	------------	------------	------------

$0+11 < \infty$
 $6+15 < \infty$
 $3+11 < \infty$

rel 3

	-	15/1	-	$\infty/0$	9/3	20/3
--	---	------	---	------------	-----	------

$11+8 < \infty$
 $11+9 < \infty$

rel 2

	-	-	-	18/2	19/3	17/2
--	---	---	---	------	------	------

$15+4 < \infty$
 $15+2 < 20$

rel 6

	-	-	-	18/2	19/3	-
--	---	---	---	------	------	---

$17+2 < 19$

rel 4

	-	-	-	-	19/3	-
--	---	---	---	---	------	---

rel 5

	-	-	-	-	-	-
--	---	---	---	---	---	---

BELLMAN - FORD

```
for  $u \in V$   
   $d[u] = \infty$ ;  $total[u] = 0$ 
```

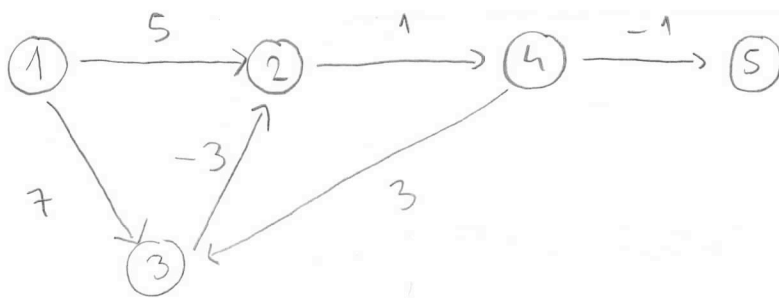
$d[s] = 0$

$O(n \cdot m)$

```
for  $k = 1, n-1$   
  for fiare  $w \in E$   
    if  $d[u] + w(u, v) < d[v]$   
       $d[v] = d[u] + w(u, v)$   
       $total[v] = u$ 
```

CICU NEGATIV

```
for fiare  $w \in E$   
  if  $d[u] + w(u, v) < d[v]$   
     $d[v] = d[u] + w(u, v)$   
     $total[v] = u$   
    STOP  $\rightarrow$  avem ciclu negativ  
    dupa  $n-1$  parcureri
```

K=1

dijkstra

1	2	3	4	5
0/0	∞/0	∞/0	∞/0	∞/0
	5/1	7/1	6/2	5/4
	4/3			

$$(1,2) \rightarrow 0+5 < \infty$$

$$(1,3) \rightarrow 0+7 < \infty$$

$$(2,4) \rightarrow 5+1 < \infty$$

$$(3,2) \rightarrow 7-3 < 5$$

$$(4,3) \rightarrow 6+3 < 7 \text{ NU}$$

$$(4,5) \rightarrow 6-1 < \infty$$

K=2

1	2	3	4	5
0/0	4/3	7/1	5/2	5/4
			5/2	4/4

1 → nu s-a actualizat
→ ignoram

$$(2,4) \rightarrow 4+1 < 6$$

$$(3,2) \rightarrow 7-3 < 4 \text{ NU}$$

$$(4,5) \rightarrow 5-1 < 5$$

$$(4,3) \rightarrow 5+3 < 7 \text{ NU}$$

K=3

1	2	3	4	5
0/0	4/3	7/1	5/2	4/4

STOP

FLOYD WARSHALL

- orole pot fi negative
- vara cicluri negative

```

for i = 1, n
  for j = 1, n
    d[i][j] = w[i][j]
    if w[i][j] = ∞
      p[i][j] = 0
    else
      p[i][j] = i
  
```

```

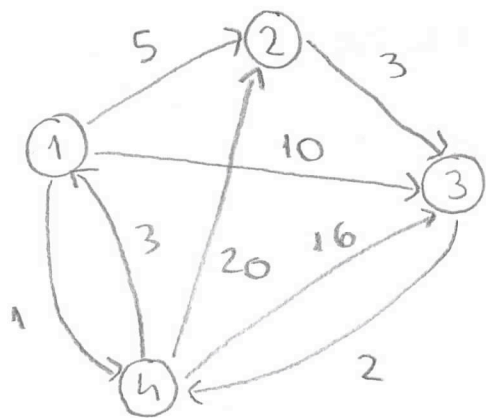
for k = 1, n // varful intermediare
  for i = 1, n
    for j = 1, n
      if ( d[i][j] > d[i][k] + d[k][j] )
        d[i][j] = d[i][k] + d[k][j]
        p[i][j] = p[k][j]
  
```

ROY - WARSHALL

$$d[i][j] = d[i][j] \text{ sau } (d[i][k] + d[k][j])$$

includere tranzitiei

↑
n



	1	2	3	4
1	0	5	10	1
2	∞	0	3	∞
3	∞	∞	0	2
4	3	20	16	0

K = 1

	1	2	3	4
1	0	5	10	1
2	∞	0	3	∞
3	∞	∞	0	2
4	3	8	13	0

K = 2

	1	2	3	4
1	0	5	8	1
2	∞	0	3	∞
3	∞	∞	0	2
4	3	8	11	0

K = 3

	1	2	3	4
1	0	5	8	1
2	∞	0	3	5
3	∞	∞	0	2
4	3	8	11	0

K = 4

	1	2	3	4
1	0	5	8	1
2	8	0	3	5
3	5	10	0	2
4	3	8	11	0

FORD - FULKERSON

FLUX MAXIM

cat timp $\exists$ un s-t lant nesat
[
 det lantul P
 reinqreste lantul P
]

EDMOND - KARP

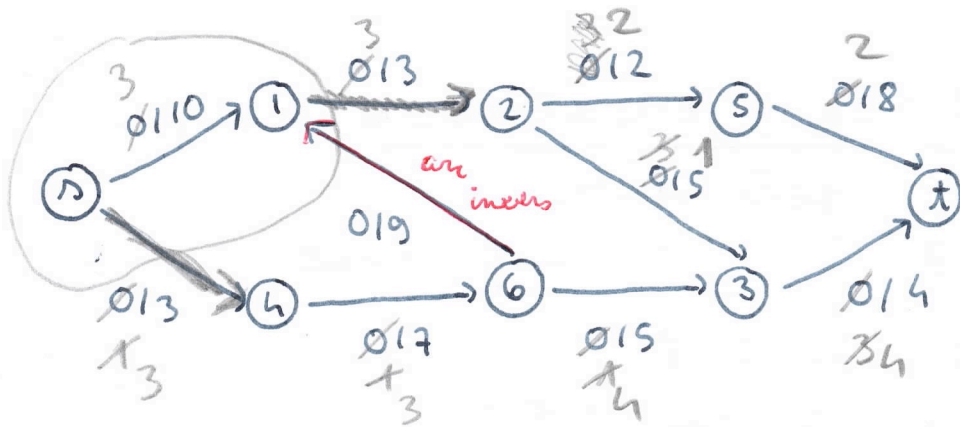
inhalizeaza fluxul cu nul

bool construneste - s-t - lant - nesat - BF
[
 parcurgem BF de la s la t doar
 lanturi cu flux & capacitate
]

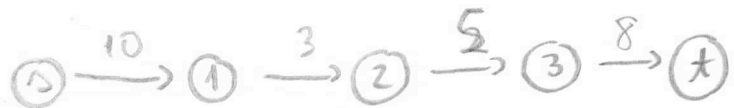
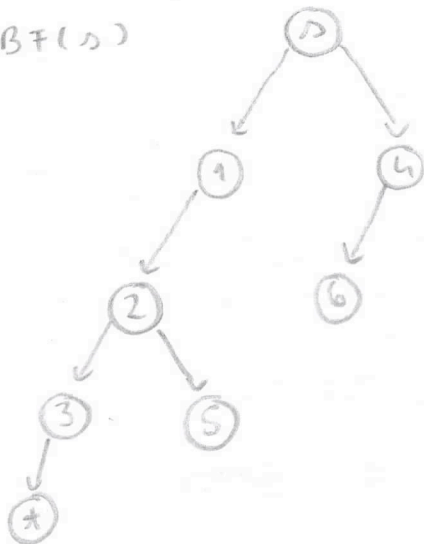
reinqreste - flux - lant
[
 fie P un s-t lant nesat
 calculam $i(P)$
 pentru $e \in P$
 [
 restem cu $i(P)$ daca $e =$ arc direct
 scadem cu $i(P)$ daca $e =$ arc invers
]
]

cat timp (construneste - lant - BF == true)
[
 reinqreste flux - lant
]

afis flux



BFS(s)



$$i(P) = 3$$

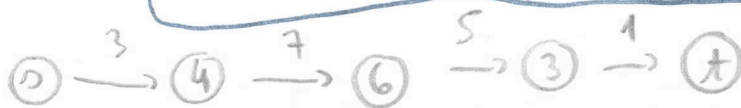
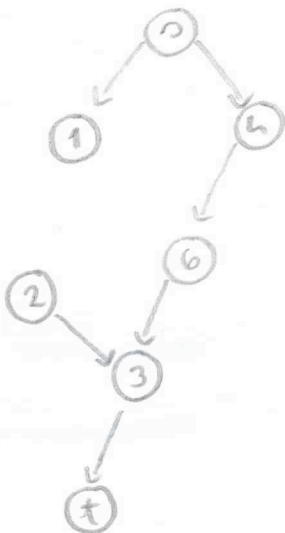
TAIETURA

$$K = \{ (s, 1), \{ 2, 3, 4, 5, 6, t \} \}$$

$$C(K) = \text{val flux} - \max = 6$$

laeluna min = of axes. din s next

BFS(s)

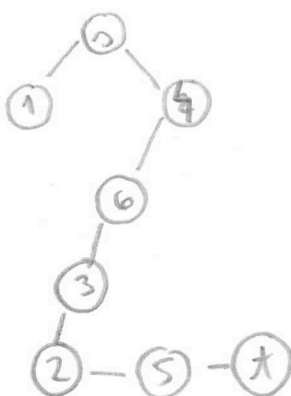


$$i(P) = 1$$



$$i(P) = 2$$

BFS(s)



BFS(s)



STOP

$$\text{FLUX MAX} = 6$$

GRAFURI EULERIENE

ciclul eulerian al lui $G =$ ciclul în G cu

$$E(C) = E(G) \quad // \text{ conține toate muchiile}$$

G eulerian $\rightarrow$ conține un ciclul eulerian

G eulerian $\Leftrightarrow$ orice vârf din G are grad par

ALGORITMUL LUI HIERHOLZER

```
euler (nod  $v$ )  
[  
    while  $d(v) > 0$   
    [  
        alege  $v, w$  muchie incidentă din  $v$   
        șterge muchia  $v, w$   
        euler ( $w$ )  
    ]  
     $C = C + v$  // adăugăm  $v$  la ciclul
```

$C = \emptyset$

euler (v)

Teorema lui Euler

G are lant euler $\Leftrightarrow G$ are cel mult două
vârfuri de grad impar

GRAFURI EULERIENE ORIENTATE

Teorema lui Euler

↳ eulerian $\Leftrightarrow \forall v \in V \quad d^-(v) = d^+(v)$

GRAFURI HAMILTONIENE

ciclul hamiltonian = ciclul în G în

$V(C) = V(G)$ // conține toate vârfurile

Condiții necesare

1. G este biconex (nu are noduri critice), în tot ^{în} ^{mediu}
nod critic = dacă eliminăm nodul, graful
nu mai este conex

Condiții suficiente

1. Th. Dirac $n \geq 3$

$\forall v \in V, d(v) \geq n/2 \Rightarrow G$ ham

2. Th. Ore $n \geq 3$

dacă $d(x) + d(y) \geq n, \forall x, y$ nedivizibile
 $\Rightarrow G$ ham